

Natural Language Processing — Complete Notes

Theory, Mathematics, Code and Examples

Contents

1	1. Introduction to NLP	4
1.1	1.1 Why NLP is hard	4
1.2	1.2 Applications of NLP	4
1.3	1.3 Levels of NLP Analysis	4
2	2. The NLP Pipeline	5
2.1	2.1 Setting up the environment	5
3	3. Text Preprocessing	6
3.1	3.1 Tokenization	6
3.1.1	3.1.1 Word Tokenization — NLTK	6
3.1.2	3.1.2 Tokenization — spaCy	6
3.1.3	3.1.3 Tokenization with plain Regex (no library)	6
3.1.4	3.1.4 Subword Tokenization (used by Transformers)	6
3.2	3.2 Lowercasing & Cleaning	7
3.3	3.3 Stop-word Removal	7
3.4	3.4 Stemming	7
3.5	3.5 Lemmatization	8
4	4. Regular Expressions in NLP	9
5	5. Bag of Words (BoW)	10
6	6. TF-IDF (Term Frequency - Inverse Document Frequency)	11
6.1	6.1 Math	11
6.2	6.2 Code	11
7	7. N-grams and Statistical Language Models	13
7.1	7.1 Generating n-grams	13
7.2	7.2 Math: probability of a sentence	13
7.3	7.3 Perplexity — evaluating a language model	14
7.4	7.4 Code: simple bigram language model	14
8	8. Part-of-Speech (POS) Tagging	15
9	9. Named Entity Recognition (NER)	16
10	10. Parsing	17
10.1	10.1 Constituency Parsing	17
10.2	10.2 Dependency Parsing	17

11.11. Word Embeddings	18
11.111.1 One-Hot Encoding (baseline, for contrast)	18
11.211.2 Word2Vec	18
11.2.1 CBOW (Continuous Bag of Words)	18
11.2.2 Skip-gram	18
11.2.3 Code — training Word2Vec with gensim	19
11.2.4 Using pretrained embeddings	19
11.2.5 Cosine similarity (used to compare embeddings)	19
11.311.3 GloVe (Global Vectors)	19
11.411.4 FastText (subword-aware embeddings)	20
11.511.5 Contextual Embeddings	20
12.12. Text Classification	21
12.112.1 Naive Bayes Classifier — Math	21
12.1.1 Code — Naive Bayes for sentiment classification	21
12.212.2 Logistic Regression for text	22
12.312.3 SVM for text classification	22
13.13. Sequence Models — RNN, LSTM, GRU	23
13.113.1 Vanilla RNN — Math	23
13.213.2 LSTM (Long Short-Term Memory)	23
13.313.3 GRU (Gated Recurrent Unit)	23
13.413.4 Code — Text classification with LSTM (PyTorch)	23
13.4.1 Bidirectional LSTM (reads sequence forward AND backward)	24
13.4.2 Keras/TensorFlow version	24
13.513.5 Seq2Seq (Encoder-Decoder)	25
14.14. Attention Mechanism	26
14.114.1 Math	26
14.214.2 Self-Attention (used in Transformers)	26
14.2.1 Code — self-attention from scratch (NumPy)	26
14.314.3 Multi-Head Attention	27
15.15. The Transformer Architecture	28
15.115.1 Architecture Overview	28
15.215.2 Positional Encoding	28
15.315.3 Feed-Forward Network (per position)	29
15.415.4 Residual Connections + Layer Norm	29
15.515.5 Building a mini-Transformer encoder layer (PyTorch)	29
16.16. BERT, GPT and Modern Pretrained Models	30
16.116.1 BERT — Masked Language Modeling (MLM)	30
16.216.2 GPT — Autoregressive (Causal) Language Modeling	30
16.316.3 Using pretrained models with HuggingFace transformers	30
16.3.1 Text classification / sentiment	30
16.3.2 Getting BERT embeddings manually	30
16.3.3 Fine-tuning BERT for classification	31
16.3.4 Text generation (GPT-2 style)	31
16.3.5 Question Answering	31
16.3.6 Summarization & Translation	31
17.17. Topic Modeling — Latent Dirichlet Allocation (LDA)	32
17.117.1 Math (generative story)	32
17.217.2 Code — LDA with gensim	32
17.2.1 Using scikit-learn	32

18.18. Evaluation Metrics	34
18.118.1 Classification Metrics	34
18.218.2 BLEU Score (Machine Translation)	34
18.318.3 ROUGE Score (Summarization)	35
18.418.4 Perplexity (Language Models)	35
19.19. End-to-End Example: Sentiment Analysis Pipeline	36
20.20. Library Cheat-Sheet — Key Functions Reference	38
20.1NLTK	38
20.2spaCy	38
20.3scikit-learn	38
20.4gensim	38
20.5transformers (HuggingFace)	39
21.21. Summary — Choosing the Right Tool	40
21.1Key Takeaways	40
22.22. Practice Exercises	41

1 1. Introduction to NLP

Natural Language Processing (NLP) is the branch of Artificial Intelligence that gives machines the ability to read, understand, interpret and generate human language (text and speech).

1.1 1.1 Why NLP is hard

Human language is:

- **Ambiguous** — “I saw the man with the telescope” (who has the telescope?)
- **Context-dependent** — meaning changes with context (“bank” of a river vs. a “bank” for money)
- **Ever-evolving** — new slang, abbreviations
- **Irregular** — exceptions to grammar rules everywhere

1.2 1.2 Applications of NLP

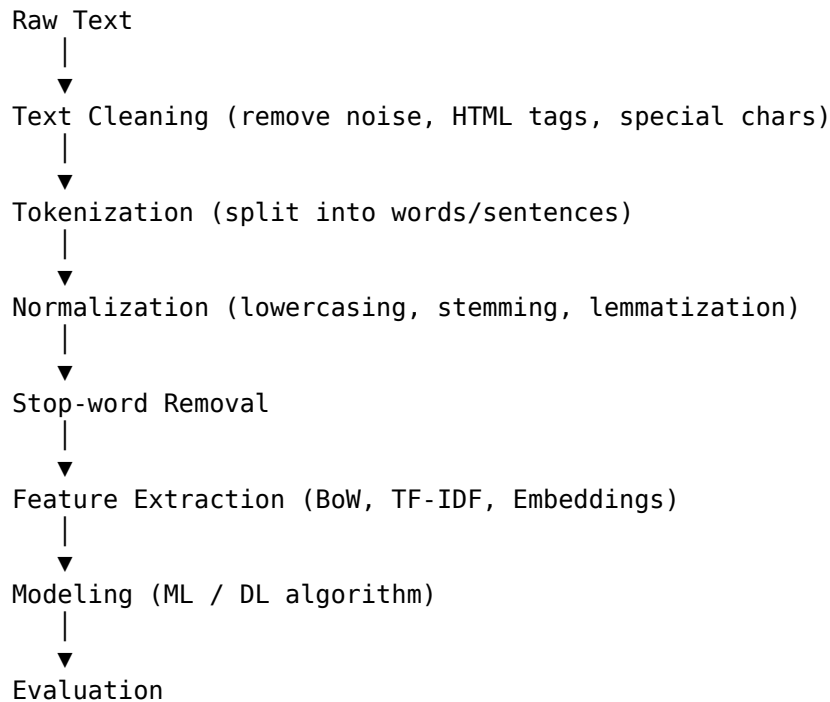
Application	Example
Machine Translation	Google Translate
Sentiment Analysis	Product review polarity
Chatbots / Virtual Assistants	Siri, Alexa
Text Summarization	News summarizers
Information Extraction	Extracting names, dates from text
Speech Recognition	Voice-to-text
Question Answering	Search engines
Spam Detection	Email filters
Text Generation	GPT-based writing assistants

1.3 1.3 Levels of NLP Analysis

1. **Phonological** — sound structure (relevant to speech)
2. **Morphological** — structure of words (prefixes, suffixes, roots)
3. **Lexical** — meaning of individual words
4. **Syntactic** — grammatical structure of sentences (parsing)
5. **Semantic** — literal meaning of sentences
6. **Discourse** — meaning across multiple sentences
7. **Pragmatic** — meaning in context / real-world knowledge

2. The NLP Pipeline

A typical NLP system follows this pipeline:



2.1 Setting up the environment

```
pip install nltk spacy scikit-learn gensim transformers torch pandas numpy matplotlib
python -m spacy download en_core_web_sm
python -c "import nltk; nltk.download('all')"
```

Key libraries used throughout these notes:

Library	Purpose
nltk	Classic NLP toolkit — tokenization, stemming, POS tagging, corpora
spaCy	Industrial-strength NLP — fast tokenization, NER, dependency parsing
scikit-learn	ML models, BoW/TF-IDF vectorizers, classifiers, metrics
gensim	Topic modeling (LDA), Word2Vec, Doc2Vec
transformers (HuggingFace)	Pretrained deep learning models (BERT, GPT, etc.)
torch / tensorflow	Deep learning backends
re	Regular expressions (built into Python)

3. Text Preprocessing

Preprocessing converts messy raw text into a clean, structured form a model can consume.

3.1 Tokenization

Tokenization is the process of splitting text into smaller units called *tokens* (words, sub-words, or sentences).

3.1.1 Word Tokenization — NLTK

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize

text = "Dr. Smith went to Washington. He arrived at 5 p.m. sharp!"

words = word_tokenize(text)
print(words)
# ['Dr.', 'Smith', 'went', 'to', 'Washington', '.', 'He', 'arrived', 'at', '5', 'p.m.', 'sharp']

sentences = sent_tokenize(text)
print(sentences)
# ['Dr. Smith went to Washington.', 'He arrived at 5 p.m. sharp!']
```

3.1.2 Tokenization — spaCy

```
import spacy
nlp = spacy.load("en_core_web_sm")

doc = nlp("Dr. Smith went to Washington. He arrived at 5 p.m. sharp!")
tokens = [token.text for token in doc]
print(tokens)

for sent in doc.sents:
    print(sent.text)
```

3.1.3 Tokenization with plain Regex (no library)

```
import re

text = "NLP is amazing! Isn't it?"
tokens = re.findall(r"\b\w+\b", text)
print(tokens)
# ['NLP', 'is', 'amazing', 'Isn', 't', 'it']
```

3.1.4 Subword Tokenization (used by Transformers)

Modern deep models (BERT, GPT) use **subword tokenization** so rare words are broken into known pieces, e.g. Byte Pair Encoding (BPE) or WordPiece.

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
tokens = tokenizer.tokenize("Unbelievably fast tokenization")
print(tokens)
# ['unbelievably', 'fast', 'token', '##ization']
```

Math intuition (BPE): Start with a vocabulary of individual characters. Iteratively merge the *most frequent adjacent symbol pair* into a new symbol, for a fixed number of merges k . This yields a vocabulary that balances word-level meaning and character-level flexibility for unseen words.

3.2 3.2 Lowercasing & Cleaning

```
import re

def clean_text(text):
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", "", text)
    text = re.sub(r"<.*?>", "", text)
    text = re.sub(r"[^a-z\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

print(clean_text("Visit https://openai.com NOW!! <b>Great</b> deal :))")
# 'visit now great deal'
```

3.3 3.3 Stop-word Removal

Stop words are very common words (the, is, at, which, on) that usually carry little discriminative meaning and are removed to reduce noise/dimensionality.

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words("english"))

words = ["this", "is", "a", "sample", "sentence", "showing", "stopword", "removal"]
filtered = [w for w in words if w.lower() not in stop_words]
print(filtered)
# ['sample', 'sentence', 'showing', 'stopword', 'removal']
```

With spaCy:

```
doc = nlp("this is a sample sentence showing stopword removal")
filtered = [token.text for token in doc if not token.is_stop]
print(filtered)
```

3.4 3.4 Stemming

Stemming chops word endings using heuristic rules to reduce a word to its *stem* (may not be a real word).

Porter Stemmer algorithm applies a sequence of rewrite rules, e.g.:

$$\text{SSES} \rightarrow \text{SS}, \quad \text{IES} \rightarrow \text{I}, \quad \text{ATIONAL} \rightarrow \text{ATE}$$

```

from nltk.stem import PorterStemmer, SnowballStemmer

ps = PorterStemmer()
words = ["running", "runs", "ran", "easily", "fairly", "studies"]
print([ps.stem(w) for w in words])
# ['run', 'run', 'ran', 'easili', 'fairli', 'studi']

sb = SnowballStemmer("english") # improved, multi-language stemmer
print([sb.stem(w) for w in words])

```

3.5 3.5 Lemmatization

Lemmatization reduces a word to its dictionary base form (*lemma*) using vocabulary + morphological analysis — always produces a real word, unlike stemming.

```

from nltk.stem import WordNetLemmatizer
lemmatizer = WordNetLemmatizer()

print(lemmatizer.lemmatize("running", pos="v")) # 'run'
print(lemmatizer.lemmatize("better", pos="a")) # 'good'
print(lemmatizer.lemmatize("studies")) # 'study'

```

With spaCy (automatically POS-aware):

```

doc = nlp("The children were running and studies were better organized")
print([token.lemma_ for token in doc])

```

Stemming vs Lemmatization

	Stemming	Lemmatization
Speed	Fast (rule-based)	Slower (dictionary lookup)
Output	May not be a real word	Always a real word
Accuracy	Lower	Higher
Example	"studies" → "studi"	"studies" → "study"

4 4. Regular Expressions in NLP

Regex is used heavily for pattern-based cleaning and extraction.

```
import re

text = "Contact us at support@example.com or sales@example.org, call +91-9876543210"

emails = re.findall(r"[\w.+-]+@[\w-]+\.[\w.-]+", text)
phones = re.findall(r"\+?\d{8,}\d", text)

print(emails)  # ['support@example.com', 'sales@example.org']
print(phones)  # ['+91-9876543210']
```

Common regex tokens: \d digit, \w word char, \s whitespace, + one-or-more, * zero-or-more, ? optional, {m,n} repetition range, ^/\$ anchors, () groups, | alternation.

5. Bag of Words (BoW)

BoW represents a document as an unordered collection (“bag”) of its word counts, ignoring grammar and word order.

Math: Given a vocabulary $V = \{w_1, w_2, \dots, w_n\}$, document d is represented as a vector

$$\vec{d} = (c_1, c_2, \dots, c_n)$$

where c_i = number of times word w_i occurs in d .

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "I love NLP and machine learning",
    "I love deep learning",
    "NLP is a subfield of AI"
]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)

print(vectorizer.get_feature_names_out())
# ['ai' 'and' 'deep' 'is' 'learning' 'love' 'machine' 'nlp' 'of' 'subfield']
print(X.toarray())
# [[0 1 0 0 1 1 1 1 0 0]
#  [0 0 1 0 1 1 0 0 0 0]
#  [1 0 0 1 0 0 0 1 1 1]]
```

Limitation: loses word order (“dog bites man” == “man bites dog”), and huge sparse vectors for large vocabularies.

6. TF-IDF (Term Frequency - Inverse Document Frequency)

TF-IDF weighs a word by how important it is to a document *relative to the whole corpus* — common words across all documents get down-weighted.

6.1 Math

Term Frequency (how often term t appears in document d):

$$\text{TF}(t, d) = \frac{\text{count of } t \text{ in } d}{\text{total terms in } d}$$

Inverse Document Frequency (how rare term t is across corpus D with N documents):

$$\text{IDF}(t, D) = \log \left(\frac{N}{1 + \text{DF}(t)} \right)$$

where $\text{DF}(t)$ = number of documents containing t (the $+1$ avoids division by zero).

TF-IDF score:

$$\text{TFIDF}(t, d, D) = \text{TF}(t, d) \times \text{IDF}(t, D)$$

A high TF-IDF score means the word occurs frequently in this document but rarely elsewhere — making it a good distinguishing keyword.

6.2 Code

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "I love NLP and machine learning",
    "I love deep learning",
    "NLP is a subfield of AI"
]

tfidf = TfidfVectorizer()
X = tfidf.fit_transform(corpus)

import pandas as pd
df = pd.DataFrame(X.toarray(), columns=tfidf.get_feature_names_out())
print(df.round(2))
```

```
# Manual TF-IDF (for understanding)
import numpy as np
from collections import Counter

def compute_tf(doc):
    words = doc.split()
    counts = Counter(words)
    return {w: c/len(words) for w, c in counts.items()}

def compute_idf(corpus):
```

```

N = len(corpus)
df = Counter()
for doc in corpus:
    for w in set(doc.split()):
        df[w] += 1
    return {w: np.log(N / (1 + c)) for w, c in df.items()}

corpus = ["nlp is fun", "nlp is powerful", "deep learning is powerful"]
idf = compute_idf(corpus)
for doc in corpus:
    tf = compute_tf(doc)
    tfidf_scores = {w: round(tf[w]*idf[w], 3) for w in tf}
    print(tfidf_scores)

```

7. N-grams and Statistical Language Models

An **n-gram** is a contiguous sequence of n tokens. A **language model** assigns a probability to a sequence of words.

7.1 Generating n-grams

```
from nltk import ngrams

text = "I love natural language processing"
tokens = text.split()

bigrams = list(ngrams(tokens, 2))
trigrams = list(ngrams(tokens, 3))
print(bigrams)    # [('I', 'love'), ('love', 'natural'), ...]
print(trigrams)

from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer(ngram_range=(1,2))    # unigrams + bigrams
X = vectorizer.fit_transform(["I love NLP", "NLP is fun"])
print(vectorizer.get_feature_names_out())
```

7.2 Math: probability of a sentence

By the chain rule, the probability of a word sequence $w_1, w_2, \dots, w_m$ is:

$$P(w_1, \dots, w_m) = \prod_{i=1}^m P(w_i \mid w_1, \dots, w_{i-1})$$

This is intractable for long sequences, so we make the **Markov assumption**: a word depends only on the previous $n - 1$ words.

Bigram model ($n = 2$):

$$P(w_i \mid w_1, \dots, w_{i-1}) \approx P(w_i \mid w_{i-1})$$

$$P(w_1, \dots, w_m) \approx \prod_{i=1}^m P(w_i \mid w_{i-1})$$

Estimating probabilities (Maximum Likelihood Estimation) from a corpus using counts C :

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i)}{C(w_{i-1})}$$

Laplace (add-one) smoothing to handle unseen bigrams:

$$P_{\text{Laplace}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}, w_i) + 1}{C(w_{i-1}) + V}$$

where V is vocabulary size.

7.3 7.3 Perplexity — evaluating a language model

Perplexity measures how “surprised” a model is by test data; lower is better.

$$PP(W) = P(w_1, w_2, \dots, w_N)^{-\frac{1}{N}} = \sqrt[N]{\prod_{i=1}^N \frac{1}{P(w_i | w_{i-1})}}$$

7.4 7.4 Code: simple bigram language model

```
from collections import defaultdict, Counter
import random

corpus = "I love NLP I love deep learning I love AI".split()

bigram_counts = defaultdict(Counter)
for w1, w2 in zip(corpus[:-1], corpus[1:]):
    bigram_counts[w1][w2] += 1

def next_word_prob(w1):
    total = sum(bigram_counts[w1].values())
    return {w2: c/total for w2, c in bigram_counts[w1].items()}

print(next_word_prob("I"))
# {'love': 1.0}
print(next_word_prob("love"))
# {'NLP': 0.333, 'deep': 0.333, 'AI': 0.333}

# Generate text
def generate(start, n=5):
    word = start
    result = [word]
    for _ in range(n):
        probs = next_word_prob(word)
        if not probs:
            break
        word = random.choices(list(probs.keys()), weights=probs.values())[0]
        result.append(word)
    return " ".join(result)

print(generate("I"))
```

8. Part-of-Speech (POS) Tagging

POS tagging assigns a grammatical category (noun, verb, adjective, etc.) to each token.

```
import nltk
tokens = nltk.word_tokenize("The quick brown fox jumps over the lazy dog")
tags = nltk.pos_tag(tokens)
print(tags)
# [('The', 'DT'), ('quick', 'JJ'), ('brown', 'JJ'), ('fox', 'NN'),
#  ('jumps', 'VBZ'), ('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN')]

doc = nlp("The quick brown fox jumps over the lazy dog")
for token in doc:
    print(token.text, token.pos_, token.tag_)
# The DET DT | quick ADJ JJ | fox NOUN NN | jumps VERB VBZ ...
```

Common tags: NN noun, VB verb, JJ adjective, RB adverb, DT determiner, IN preposition, PRP pronoun.

Math (behind statistical POS tagging — Hidden Markov Model): Find the tag sequence $\hat{t}_1^n$ maximizing

$$\hat{t}_1^n = \arg \max_{t_1^n} \prod_{i=1}^n P(w_i | t_i) P(t_i | t_{i-1})$$

solved efficiently with the **Viterbi algorithm** (dynamic programming).

9. Named Entity Recognition (NER)

NER identifies and classifies real-world entities (Person, Organization, Location, Date, etc.) in text.

```
doc = nlp("Apple Inc. was founded by Steve Jobs in Cupertino on April 1, 1976.")
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
# Apple Inc. -> ORG
# Steve Jobs -> PERSON
# Cupertino -> GPE
# April 1, 1976 -> DATE
```

```
from spacy import displacy
displacy.render(doc, style="ent")  # visualize entities (in Jupyter/HTML)
```

Using NLTK (chunking-based):

```
import nltk
sentence = "Apple Inc. was founded by Steve Jobs in Cupertino"
tokens = nltk.word_tokenize(sentence)
tags = nltk.pos_tag(tokens)
tree = nltk.ne_chunk(tags)
print(tree)
```

Using HuggingFace transformer-based NER (state of the art):

```
from transformers import pipeline

ner = pipeline("ner", grouped_entities=True)
result = ner("Apple Inc. was founded by Steve Jobs in Cupertino.")
print(result)
```


10 10. Parsing

10.1 10.1 Constituency Parsing

Breaks a sentence into nested sub-phrases (NP, VP, etc.) following a grammar.

```
import nltk
grammar = nltk.CFG.fromstring("""
    S -> NP VP
    NP -> Det N
    VP -> V NP
    Det -> 'the'
    N -> 'dog' | 'cat'
    V -> 'chased'
""")
parser = nltk.ChartParser(grammar)
for tree in parser.parse(['the', 'dog', 'chased', 'the', 'cat']):
    print(tree)
    tree.pretty_print()
```

10.2 10.2 Dependency Parsing

Represents grammatical relations between words as a tree of head→dependent links — this is what modern pipelines (spaCy) use by default.

```
doc = nlp("The dog chased the cat")
for token in doc:
    print(f"{token.text:10} --{token.dep_:10}--> {token.head.text}")

displacy.render(doc, style="dep")    # visual dependency tree
```

11 11. Word Embeddings

Embeddings represent words as **dense, low-dimensional real-valued vectors** that capture semantic meaning, unlike sparse one-hot / BoW vectors. Words with similar meaning end up close together in vector space.

11.1 11.1 One-Hot Encoding (baseline, for contrast)

Each word is a vector of size $|V|$ with a single 1:

$$\text{"cat"} = [0, 0, 1, 0, \dots, 0]$$

Problem: all words are equidistant — no notion of similarity, and vectors are huge & sparse.

11.2 11.2 Word2Vec

Word2Vec (Mikolov et al., 2013) learns embeddings by training a shallow neural network to predict context from a word (**Skip-gram**) or a word from its context (**CBOW**).

11.2.1 CBOW (Continuous Bag of Words)

Predicts the **target word** from surrounding **context words**.

$$P(w_t \mid w_{t-c}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+c})$$

11.2.2 Skip-gram

Predicts **context words** from the **target word** (better for rare words).

$$P(w_{t+j} \mid w_t) \quad \text{for } j \in [-c, c], j \neq 0$$

Objective (Skip-gram): maximize average log probability

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} \mid w_t)$$

where, using softmax over the whole vocabulary:

$$P(w_O \mid w_I) = \frac{\exp(v'_{w_O}{}^\top v_{w_I})}{\sum_{w=1}^V \exp(v'_w{}^\top v_{w_I})}$$

v_{w_I} = “input” embedding of the center word, v'_{w_O} = “output” embedding of the context word. Because the softmax denominator over the full vocabulary is expensive, Word2Vec uses **Negative Sampling** or **Hierarchical Softmax** to approximate it efficiently.

Negative Sampling objective:

$$\log \sigma(v'_{w_O}{}^\top v_{w_I}) + \sum_{i=1}^k \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-v'_{w_i}{}^\top v_{w_I})]$$

(σ = sigmoid; k negative “noise” words are sampled per positive pair, so the model just learns to distinguish real context words from random ones — far cheaper than full softmax.)

11.2.3 Code — training Word2Vec with gensim

```
from gensim.models import Word2Vec

sentences = [
    ["i", "love", "natural", "language", "processing"],
    ["i", "love", "deep", "learning"],
    ["nlp", "is", "a", "subfield", "of", "ai"],
]

model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, sg=1) # sg=1 -> skip-gram
model.train(sentences, total_examples=len(sentences), epochs=50)

print(model.wv["love"]) # 100-dim embedding vector
print(model.wv.most_similar("love", topn=3)) # nearest neighbours
print(model.wv.similarity("love", "learning"))
```

11.2.4 Using pretrained embeddings

```
import gensim.downloader as api
wv = api.load("word2vec-google-news-300") # 300-dim, trained on Google News

print(wv.most_similar("king", topn=5))
print(wv.similarity("man", "woman"))

# Famous analogy: king - man + woman = queen
result = wv.most_similar(positive=["king", "woman"], negative=["man"], topn=1)
print(result) # [('queen', 0.71...)]
```

Math behind the analogy: vector arithmetic works because directions in embedding space encode relationships:

$$v_{\text{king}} - v_{\text{man}} + v_{\text{woman}} \approx v_{\text{queen}}$$

11.2.5 Cosine similarity (used to compare embeddings)

$$\text{cosine_sim}(A, B) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_i A_i B_i}{\sqrt{\sum_i A_i^2} \sqrt{\sum_i B_i^2}}$$

```
import numpy as np
def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

11.3 GloVe (Global Vectors)

GloVe learns embeddings from **global word-word co-occurrence statistics** rather than local context windows.

Let X_{ij} = number of times word j occurs in the context of word i . GloVe minimizes:

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(v_i^\top \tilde{v}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

where $f(X_{ij})$ is a weighting function that down-weights very frequent co-occurrences.

```
# Loading pretrained GloVe vectors
import gensim.downloader as api
glove = api.load("glove-wiki-gigaword-100")
print(glove.most_similar("computer"))
```

11.4 FastText (subword-aware embeddings)

FastText represents each word as a bag of **character n-grams**, so it can produce embeddings even for out-of-vocabulary words (great for morphologically rich languages).

```
from gensim.models import FastText
model = FastText(sentences, vector_size=100, window=5, min_count=1, sg=1)
print(model.wv["processing"]) # works even for unseen/misspelled words
```

11.5 Contextual Embeddings

Word2Vec/GloVe give **one fixed vector per word** regardless of context (“bank” of river vs. money bank have the same vector). Contextual models like **ELMo**, **BERT**, **GPT** produce a *different* embedding for the same word depending on the sentence — covered in Section 14.

12 12. Text Classification

12.1 12.1 Naive Bayes Classifier — Math

Naive Bayes applies Bayes' theorem with a "naive" assumption that features (words) are conditionally independent given the class.

Bayes' theorem:

$$P(c | d) = \frac{P(d | c) P(c)}{P(d)}$$

For classification we only need the numerator (denominator is constant across classes):

$$\hat{c} = \arg \max_{c \in C} P(c) \prod_{i=1}^n P(w_i | c)$$

With **Laplace smoothing** to avoid zero probabilities for unseen words:

$$P(w_i | c) = \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} \text{count}(w, c) + |V|}$$

Log-space computation (to avoid numerical underflow from multiplying many small probabilities):

$$\hat{c} = \arg \max_c \left[\log P(c) + \sum_{i=1}^n \log P(w_i | c) \right]$$

12.1.1 Code — Naive Bayes for sentiment classification

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report

texts = ["I love this movie", "This film was terrible", "Amazing acting and story",
         "Worst movie ever", "I really enjoyed it", "Boring and too long"]
labels = [1, 0, 1, 0, 1, 0] # 1 = positive, 0 = negative

X_train, X_test, y_train, y_test = train_test_split(texts, labels, test_size=0.3, random_state=42)

vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

model = MultinomialNB()
model.fit(X_train_vec, y_train)

preds = model.predict(X_test_vec)
print("Accuracy:", accuracy_score(y_test, preds))
print(classification_report(y_test, preds))
```

12.2 12.2 Logistic Regression for text

$$P(y = 1 \mid \vec{x}) = \sigma(\vec{w}^\top \vec{x} + b) = \frac{1}{1 + e^{-(\vec{w}^\top \vec{x} + b)}}$$

Trained by minimizing cross-entropy (log) loss:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

```
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer()
X_train_vec = tfidf.fit_transform(X_train)
X_test_vec = tfidf.transform(X_test)

clf = LogisticRegression()
clf.fit(X_train_vec, y_train)
print(clf.predict(X_test_vec))
```

12.3 12.3 SVM for text classification

```
from sklearn.svm import LinearSVC
svm_clf = LinearSVC()
svm_clf.fit(X_train_vec, y_train)
print(svm_clf.predict(X_test_vec))
```

13.13. Sequence Models — RNN, LSTM, GRU

Text is sequential; **Recurrent Neural Networks (RNNs)** process one token at a time while maintaining a *hidden state* that summarizes everything seen so far.

13.1.1 13.1 Vanilla RNN — Math

At each time step t , given input x_t and previous hidden state h_{t-1} :

$$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

$$\hat{y}_t = \text{softmax}(W_{hy}h_t + b_y)$$

Problem: During backpropagation-through-time (BPTT), gradients are repeatedly multiplied by W_{hh} , causing the **vanishing/exploding gradient problem** over long sequences — the network “forgets” long-range dependencies.

13.2 13.2 LSTM (Long Short-Term Memory)

LSTM introduces a **cell state** C_t and three **gates** (forget, input, output) that regulate information flow, solving the vanishing gradient problem.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (\text{forget gate})$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (\text{input gate})$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad (\text{candidate cell state})$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad (\text{new cell state})$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (\text{output gate})$$

$$h_t = o_t \odot \tanh(C_t) \quad (\text{new hidden state})$$

$\odot$ = element-wise product, σ = sigmoid. The forget gate decides what to discard from C_{t-1} ; the input gate decides what new info to add; the output gate decides what to expose as h_t .

13.3 13.3 GRU (Gated Recurrent Unit)

A simplified LSTM with 2 gates instead of 3 (fewer parameters, often similar performance):

$$z_t = \sigma(W_z[h_{t-1}, x_t]) \quad (\text{update gate})$$

$$r_t = \sigma(W_r[h_{t-1}, x_t]) \quad (\text{reset gate})$$

$$\tilde{h}_t = \tanh(W[h_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

13.4 13.4 Code — Text classification with LSTM (PyTorch)

```

import torch
import torch.nn as nn

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim, num_classes):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, num_classes)

    def forward(self, x):
        embedded = self.embedding(x)           # (batch, seq_len, embed_dim)
        output, (h_n, c_n) = self.lstm(embedded) # h_n: (1, batch, hidden_dim)
        out = self.fc(h_n.squeeze(0))          # (batch, num_classes)
        return out

vocab_size, embed_dim, hidden_dim, num_classes = 5000, 128, 64, 2
model = LSTMClassifier(vocab_size, embed_dim, hidden_dim, num_classes)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

# dummy batch: batch_size=4, seq_len=10
x_batch = torch.randint(1, vocab_size, (4, 10))
y_batch = torch.tensor([0, 1, 1, 0])

optimizer.zero_grad()
logits = model(x_batch)
loss = criterion(logits, y_batch)
loss.backward()
optimizer.step()
print("Loss:", loss.item())

```

13.4.1 Bidirectional LSTM (reads sequence forward AND backward)

```

self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True, bidirectional=True)
# output hidden dim becomes 2*hidden_dim (forward + backward concatenated)

```

13.4.2 Keras/TensorFlow version

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

model = Sequential([
    Embedding(input_dim=5000, output_dim=128, input_length=100),
    LSTM(64),
    Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
model.summary()

```


13.5 13.5 Seq2Seq (Encoder-Decoder)

Used for translation, summarization: an **encoder** RNN compresses the input sequence into a context vector; a **decoder** RNN generates the output sequence from it.

$$\begin{aligned}\text{context} &= \text{Encoder}(x_1, \dots, x_n) = h_n \\ y_t &= \text{Decoder}(y_{t-1}, h_{t-1})\end{aligned}$$

Limitation: a single fixed-size context vector is a bottleneck for long sequences $\rightarrow$ motivates **Attention** (next section).

14 14. Attention Mechanism

Attention lets the decoder “look back” at *all* encoder hidden states and dynamically weight the most relevant ones at each decoding step, instead of relying on one fixed context vector.

14.1 14.1 Math

Given decoder state s_{t-1} and all encoder hidden states $h_1, \dots, h_n$:

1. Alignment (score) function — how well input j matches output t :

$$e_{tj} = \text{score}(s_{t-1}, h_j)$$

Common choices:

$$\text{score}(s, h) = s^\top h \quad (\text{dot product})$$

$$\text{score}(s, h) = v^\top \tanh(W_1 s + W_2 h) \quad (\text{Bahdanau/additive})$$

2. Attention weights (softmax normalizes scores into a probability distribution):

$$\alpha_{tj} = \frac{\exp(e_{tj})}{\sum_{k=1}^n \exp(e_{tk})}$$

3. Context vector — weighted sum of encoder states:

$$c_t = \sum_{j=1}^n \alpha_{tj} h_j$$

The decoder then uses c_t (along with s_{t-1}) to predict y_t .

14.2 14.2 Self-Attention (used in Transformers)

Self-attention relates different positions of a *single* sequence to compute a representation of that sequence, using three learned projections of the input X : **Query (Q)**, **Key (K)**, **Value (V)**.

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V$$

- $QK^\top$: similarity score between every pair of tokens.
- Divide by $\sqrt{d_k}$ (d_k = key dimension) to keep gradients stable (prevents dot products from growing too large).
- softmax turns scores into attention weights that sum to 1.
- Multiply by V to get a weighted combination of value vectors → the output representation.

14.2.1 Code — self-attention from scratch (NumPy)

```

import numpy as np

def softmax(x, axis=-1):
    e_x = np.exp(x - np.max(x, axis=axis, keepdims=True))
    return e_x / e_x.sum(axis=axis, keepdims=True)

def self_attention(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    d_k = K.shape[-1]
    scores = Q @ K.T / np.sqrt(d_k)
    weights = softmax(scores, axis=-1)
    output = weights @ V
    return output, weights

np.random.seed(0)
X = np.random.rand(4, 8)      # 4 tokens, embedding dim 8
Wq = np.random.rand(8, 8)
Wk = np.random.rand(8, 8)
Wv = np.random.rand(8, 8)

output, attn_weights = self_attention(X, Wq, Wk, Wv)
print("Attention weights:\n", attn_weights.round(2))
print("Output shape:", output.shape)    # (4, 8)

```

14.3 Multi-Head Attention

Instead of one attention function, run h attention “heads” in parallel on different learned projections, then concatenate — lets the model attend to different types of relationships simultaneously (e.g., syntax vs. coreference).

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

```

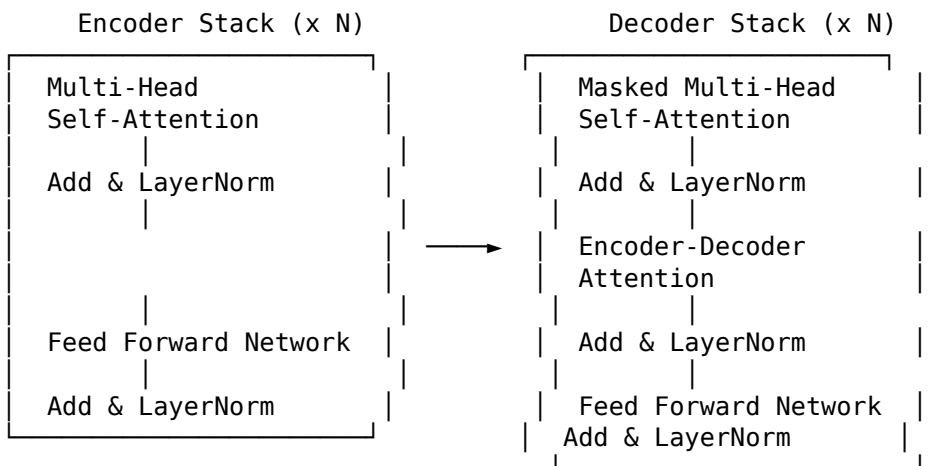
import torch, torch.nn as nn
mha = nn.MultiheadAttention(embed_dim=64, num_heads=8, batch_first=True)
x = torch.rand(2, 10, 64)    # (batch, seq_len, embed_dim)
output, attn_weights = mha(x, x, x)    # self-attention: Q=K=V=x
print(output.shape)          # torch.Size([2, 10, 64])

```

15. The Transformer Architecture

Introduced in “*Attention Is All You Need*” (Vaswani et al., 2017). Transformers dispense with recurrence entirely, relying only on attention — enabling massive parallelization and long-range dependency modeling.

15.1 Architecture Overview



15.2 Positional Encoding

Since there’s no recurrence, the model has no innate sense of token order — **positional encodings** are added to input embeddings to inject position information.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

where pos = position in sequence, i = dimension index, d_{model} = embedding dimension. Sinusoids of different frequencies let the model learn to attend by relative position.

```
import numpy as np

def positional_encoding(seq_len, d_model):
    pos = np.arange(seq_len)[:, np.newaxis]
    i = np.arange(d_model)[np.newaxis, :]
    angle_rates = 1 / np.power(10000, (2*(i//2)) / np.float32(d_model))
    angle_rads = pos * angle_rates
    angle_rads[:, 0::2] = np.sin(angle_rads[:, 0::2])
    angle_rads[:, 1::2] = np.cos(angle_rads[:, 1::2])
    return angle_rads

pe = positional_encoding(50, 128)
print(pe.shape)  # (50, 128)
```

15.3 15.3 Feed-Forward Network (per position)

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

15.4 15.4 Residual Connections + Layer Norm

Each sub-layer output is: $\text{LayerNorm}(x + \text{Sublayer}(x))$ — residual connections ease gradient flow in deep stacks; layer norm stabilizes training.

15.5 15.5 Building a mini-Transformer encoder layer (PyTorch)

```
import torch.nn as nn

encoder_layer = nn.TransformerEncoderLayer(d_model=512, nhead=8,
                                           dim_feedforward=2048, batch_first=True)
transformer_encoder = nn.TransformerEncoder(encoder_layer, num_layers=6)

x = torch.rand(32, 20, 512)  # (batch, seq_len, d_model)
out = transformer_encoder(x)
print(out.shape)  # torch.Size([32, 20, 512])
```

16 16. BERT, GPT and Modern Pretrained Models

Model	Architecture	Training objective	Typical use
BERT (2018)	Transformer <i>encoder</i> only, bidirectional	Masked Language Modeling + Next Sentence Prediction	Classification, NER, QA (understanding)
GPT (2018+)	Transformer <i>decoder</i> only, autoregressive	Predict next token (left-to-right)	Text generation, chat
T5 / BART	Full encoder-decoder	Text-to-text / denoising	Translation, summarization

16.1 16.1 BERT — Masked Language Modeling (MLM)

15% of input tokens are randomly masked; the model must predict them using **bidirectional** context (unlike GPT, which only sees left context).

$$\mathcal{L}_{MLM} = - \sum_{i \in \text{masked}} \log P(w_i \mid w_{\setminus i})$$

16.2 16.2 GPT — Autoregressive (Causal) Language Modeling

$$\mathcal{L}_{LM} = - \sum_{i=1}^n \log P(w_i \mid w_1, \dots, w_{i-1})$$

Uses **masked (causal) self-attention** so token i can only attend to tokens $1, \dots, i$ (can't "see the future").

16.3 16.3 Using pretrained models with HuggingFace transformers

16.3.1 Text classification / sentiment

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")
print(classifier("I absolutely loved this movie!"))
# [{'label': 'POSITIVE', 'score': 0.9998}]
```

16.3.2 Getting BERT embeddings manually

```
from transformers import AutoTokenizer, AutoModel
import torch

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")

inputs = tokenizer("Natural language processing is fascinating",
                  return_tensors="pt")
with torch.no_grad():
    outputs = model(**inputs)
```

```
last_hidden_state = outputs.last_hidden_state # (1, seq_len, 768)
cls_embedding = last_hidden_state[:, 0, :] # sentence-level embedding
print(cls_embedding.shape) # torch.Size([1, 768])
```

16.3.3 Fine-tuning BERT for classification

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from datasets import Dataset

data = Dataset.from_dict({
    "text": ["I love this", "I hate this", "Amazing!", "Terrible experience"],
    "label": [1, 0, 1, 0]
})

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
def tokenize_fn(batch):
    return tokenizer(batch["text"], padding=True, truncation=True)
data = data.map(tokenize_fn, batched=True)

model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

args = TrainingArguments(output_dir="./out", num_train_epochs=3,
                        per_device_train_batch_size=8, logging_steps=1)
trainer = Trainer(model=model, args=args, train_dataset=data)
trainer.train()
```

16.3.4 Text generation (GPT-2 style)

```
generator = pipeline("text-generation", model="gpt2")
result = generator("Natural language processing is", max_length=30, num_return_sequences=1)
print(result[0]["generated_text"])
```

16.3.5 Question Answering

```
qa = pipeline("question-answering")
context = "NLP stands for Natural Language Processing. It is a field of AI."
result = qa(question="What does NLP stand for?", context=context)
print(result["answer"]) # 'Natural Language Processing'
```

16.3.6 Summarization & Translation

```
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
summary = summarizer(long_text, max_length=60, min_length=20, do_sample=False)

translator = pipeline("translation_en_to_fr", model="Helsinki-NLP/opus-mt-en-fr")
print(translator("Natural language processing is amazing")[0]["translation_text"])
```

17 17. Topic Modeling — Latent Dirichlet Allocation (LDA)

LDA is a generative probabilistic model that assumes each document is a mixture of topics, and each topic is a distribution over words.

17.1 17.1 Math (generative story)

For each document d : 1. Draw topic distribution $\theta_d \sim \text{Dirichlet}(\alpha)$ 2. For each word position: - Draw a topic $z \sim \text{Multinomial}(\theta_d)$ - Draw a word $w \sim \text{Multinomial}(\phi_z)$, where $\phi_z \sim \text{Dirichlet}(\beta)$ is the word distribution for topic z

The joint probability of a document's words and topics:

$$P(W, Z, \theta, \phi \mid \alpha, \beta) = \prod_k P(\phi_k \mid \beta) \prod_d P(\theta_d \mid \alpha) \prod_n P(z_{d,n} \mid \theta_d) P(w_{d,n} \mid \phi_{z_{d,n}})$$

Inference (finding θ, ϕ, Z) is done via **Variational Bayes** or **Gibbs Sampling** — intractable in closed form.

17.2 17.2 Code — LDA with gensim

```
from gensim import corpora
from gensim.models import LdaModel

documents = [
    "cats and dogs are popular pets",
    "the stock market crashed today",
    "dogs love playing fetch in the park",
    "investors are worried about the economy",
    "my cat sleeps all day",
]

texts = [[w for w in doc.lower().split() if w not in {"and", "the", "are", "in"}] for doc in documents]
dictionary = corpora.Dictionary(texts)
corpus = [dictionary.doc2bow(text) for text in texts]

lda = LdaModel(corpus=corpus, id2word=dictionary, num_topics=2, passes=20, random_state=42)

for idx, topic in lda.print_topics(-1):
    print(f"Topic {idx}: {topic}")

# Infer topic distribution for a new document
new_doc = "my puppy loves the park"
bow = dictionary.doc2bow(new_doc.lower().split())
print(lda.get_document_topics(bow))
```

17.2.1 Using scikit-learn

```
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.feature_extraction.text import CountVectorizer
```



```
vectorizer = CountVectorizer(stop_words="english")
X = vectorizer.fit_transform(documents)

lda_sklearn = LatentDirichletAllocation(n_components=2, random_state=42)
lda_sklearn.fit(X)

feature_names = vectorizer.get_feature_names_out()
for topic_idx, topic in enumerate(lda_sklearn.components_):
    top_words = [feature_names[i] for i in topic.argsort()[-5:]]
    print(f"Topic {topic_idx}: {top_words}")
```

18.18. Evaluation Metrics

18.18.1 Classification Metrics

Given a confusion matrix with True Positives (TP), True Negatives (TN), False Positives (FP), False Negatives (FN):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (\text{of predicted positives, how many correct?})$$

$$\text{Recall (Sensitivity)} = \frac{TP}{TP + FN} \quad (\text{of actual positives, how many found?})$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

y_true = [1, 0, 1, 1, 0, 1, 0]
y_pred = [1, 0, 0, 1, 0, 1, 1]

print("Accuracy:", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall:", recall_score(y_true, y_pred))
print("F1:", f1_score(y_true, y_pred))
print("Confusion matrix:\n", confusion_matrix(y_true, y_pred))
```

18.2 BLEU Score (Machine Translation)

BLEU compares n-gram overlap between a candidate translation and reference translation(s), with a brevity penalty to discourage overly short outputs.

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right)$$

where p_n = modified n-gram precision, BP = brevity penalty:

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{(1-r/c)} & \text{if } c \leq r \end{cases}$$

(c = candidate length, r = reference length)

```
from nltk.translate.bleu_score import sentence_bleu

reference = ["the", "cat", "is", "on", "the", "mat"]
candidate = ["the", "cat", "sat", "on", "the", "mat"]
score = sentence_bleu(reference, candidate)
print("BLEU score:", score)
```

18.3 ROUGE Score (Summarization)

ROUGE-N measures n-gram **recall** overlap between generated and reference summary.

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{Ref}} \sum_{\text{gram}_n \in S} \text{Count}_{\text{match}}(\text{gram}_n)}{\sum_{S \in \text{Ref}} \sum_{\text{gram}_n \in S} \text{Count}(\text{gram}_n)}$$

```
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'], use_stemmer=True)
scores = scorer.score("the cat sat on the mat", "the cat is on the mat")
print(scores)
```

18.4 Perplexity (Language Models)

Already defined in §7.3 — lower perplexity = better model fit to test data.

```
import numpy as np
def perplexity(probs):
    # probs: list of P(w_i | context) assigned by the model to the actual test sequence
    N = len(probs)
    return np.exp(-np.sum(np.log(probs)) / N)

print(perplexity([0.2, 0.5, 0.1, 0.4]))
```

19. End-to-End Example: Sentiment Analysis Pipeline

```
import pandas as pd
import re
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

# 1. Sample data
data = pd.DataFrame({
    "review": [
        "This product is amazing, I love it!",
        "Terrible quality, complete waste of money",
        "Best purchase I have made this year",
        "Awful experience, would not recommend",
        "Absolutely fantastic service and product",
        "Very disappointed with this item"
    ],
    "sentiment": [1, 0, 1, 0, 1, 0]
})

# 2. Preprocessing
lemmatizer = WordNetLemmatizer()
stop_words = set(stopwords.words("english"))

def preprocess(text):
    text = text.lower()
    text = re.sub(r"^[a-z\s]", "", text)
    words = text.split()
    words = [lemmatizer.lemmatize(w) for w in words if w not in stop_words]
    return " ".join(words)

data["clean_review"] = data["review"].apply(preprocess)

# 3. Feature extraction
tfidf = TfidfVectorizer()
X = tfidf.fit_transform(data["clean_review"])
y = data["sentiment"]

# 4. Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 5. Model
model = LogisticRegression()
model.fit(X_train, y_train)

# 6. Evaluate
preds = model.predict(X_test)
print(classification_report(y_test, preds))
```

7. Predict new review

```
new_review = preprocess("I really enjoyed using this, works great!")  
new_vec = tfidf.transform([new_review])  
print("Prediction:", model.predict(new_vec))
```

20 20. Library Cheat-Sheet — Key Functions Reference

20.1 NLTK

Function	Purpose
<code>nltk.word_tokenize(text)</code>	Split text into words
<code>nltk.sent_tokenize(text)</code>	Split text into sentences
<code>nltk.pos_tag(tokens)</code>	POS tagging
<code>nltk.ne_chunk(tagged)</code>	Named entity chunking
<code>nltk.corpus.stopwords.words('english')</code>	Get stop-word list
<code>nltk.stem.PorterStemmer().stem(w)</code>	Stem a word
<code>nltk.stem.WordNetLemmatizer().lemmatize(w)</code>	Lemmatize a word
<code>nltk.ngrams(tokens, n)</code>	Generate n-grams
<code>nltk.translate.bleu_score.sentence_bleu()</code>	BLEU score
<code>nltk.CFG.fromstring()</code> / <code>nltk.ChartParser()</code>	Constituency parsing

20.2 spaCy

Function	Purpose
<code>spacy.load("en_core_web_sm")</code>	Load pipeline
<code>nlp(text)</code>	Process text → Doc object
<code>doc.sents</code>	Sentence segmentation
<code>token.lemma_</code> / <code>token.pos_</code> / <code>token.dep_</code>	Lemma, POS, dependency label
<code>token.is_stop</code>	Check if stop word
<code>doc.ents</code>	Named entities
<code>displacy.render(doc, style="dep"/"ent")</code>	Visualize

20.3 scikit-learn

Function	Purpose
<code>CountVectorizer()</code>	Bag-of-Words vectorizer
<code>TfidfVectorizer()</code>	TF-IDF vectorizer
<code>MultinomialNB()</code>	Naive Bayes classifier
<code>LogisticRegression()</code>	Logistic regression classifier
<code>LinearSVC()</code>	Linear SVM classifier
<code>LatentDirichletAllocation()</code>	Topic modeling
<code>train_test_split()</code>	Split data
<code>accuracy_score</code> / <code>precision_score</code> / <code>recall_score</code> / <code>f1_score</code>	Metrics
<code>confusion_matrix()</code>	Confusion matrix

20.4 gensim

Function	Purpose
<code>Word2Vec(sentences, vector_size, window, sg)</code>	Train Word2Vec
<code>model.wv.most_similar(word)</code>	Nearest neighbours

Function	Purpose
<code>model.wv.similarity(w1, w2)</code>	Cosine similarity
<code>FastText(...)</code>	Subword embeddings
<code>corpora.Dictionary(texts)</code>	Build vocabulary
<code>LdaModel(corpus, id2word, num_topics)</code>	Train LDA topic model
<code>gensim.downloader.load(name)</code>	Load pretrained vectors

20.5 transformers (HuggingFace)

Function	Purpose
<code>pipeline("sentiment-analysis")</code>	Ready-made sentiment classifier
<code>pipeline("ner", grouped_entities=True)</code>	Ready-made NER
<code>pipeline("text-generation")</code>	GPT-style generation
<code>pipeline("question-answering")</code>	Extractive QA
<code>pipeline("summarization")</code>	Text summarization
<code>pipeline("translation_xx_to_yy")</code>	Translation
<code>AutoTokenizer.from_pretrained(name)</code>	Load tokenizer
<code>AutoModel.from_pretrained(name)</code>	Load base model
<code>AutoModelForSequenceClassification.from_pretrained(name)</code>	Load classification head
<code>Trainer(model, args, train_dataset)</code>	Fine-tuning loop

21 21. Summary — Choosing the Right Tool

Task	Simple/Classical	Modern/Deep Learning
Tokenization	NLTK, regex	spaCy, HF tokenizers (BPE/WordPiece)
Text representation	BoW, TF-IDF	Word2Vec, GloVe, BERT embeddings
Classification	Naive Bayes, SVM, Logistic Regression	Fine-tuned BERT/RoBERTa
Sequence labeling (POS/NER)	HMM, CRF	spaCy pipelines, BERT-NER
Language modeling	N-gram models	RNN/LSTM, GPT
Translation/Summarization	Statistical MT	Seq2Seq + Attention, T5/BART
Topic discovery	LDA	BERTopic (embeddings + clustering)

21.1 Key Takeaways

1. **Preprocessing quality strongly affects downstream results** — tokenize, normalize, remove noise carefully.
2. **BoW/TF-IDF** are fast, interpretable baselines but ignore word order and semantics.
3. **Word embeddings** (Word2Vec, GloVe) capture semantic similarity via dense vectors trained on co-occurrence.
4. **RNN/LSTM** model sequences but struggle with very long-range dependencies and are hard to parallelize.
5. **Attention & Transformers** solve both issues — parallel computation + direct access to all positions — and underpin virtually all modern state-of-the-art NLP (BERT, GPT, T5).
6. **Always evaluate** with task-appropriate metrics: Accuracy/F1 for classification, BLEU/ROUGE for generation, Perplexity for language models.

22. Practice Exercises

1. Write a function that tokenizes a paragraph, removes stop words, lemmatizes, and returns the top 5 most frequent remaining words.
2. Implement TF-IDF from scratch (without sklearn) and verify your output matches `TfidfVectorizer`.
3. Train a Word2Vec model on a small corpus of your choice and find the 5 nearest neighbours of 3 different words.
4. Build a Naive Bayes spam classifier using a public SMS spam dataset; report precision, recall, F1.
5. Implement scaled dot-product self-attention from scratch in NumPy and verify the attention weights sum to 1 across each row.
6. Fine-tune `bert-base-uncased` on a small labeled sentiment dataset using HuggingFace Trainer, and compare accuracy against a TF-IDF + Logistic Regression baseline.
7. Compute the perplexity of a bigram language model on a held-out test sentence.
8. Use `gensim`'s `LdaModel` on a set of news headlines and interpret the resulting topics.
9. Compute BLEU score between a machine-translated sentence and a human reference translation, and explain the effect of the brevity penalty.
10. Use `spaCy`'s dependency parser to extract subject-verb-object triples from a paragraph of text.

End of Notes.